



VERIFIZIERTE C-LIBRARY FÜR LOCKFREIE DATENSTRUKTUREN

ENTWICKLUNG DER LOCKFREIEN DATENSTRUKTUREN

Software-Entwicklungspraktikum (SEP)

Sommersemester 2023

Pflichtenheft

Auftraggeber

Technische Universität Braunschweig

Institute of Theoretical Computer Science

Prof. Dr. rer. nat. Roland Meyer

IZ 346, i.e. Informatikzentrum, 3rd floor, office 346, Mühlenpfordtstr. 23

D-38106 Braunschweig

Betreuer: Sören van der Wall

Auftragnehmer:

Name	E-Mail-Adresse
Keanu Wirz	k.wirz@tu-braunschweig.de
Nazli Cesur	n.cesur@tu-braunschweig.de
Tristen Vaeckenstedt	t.vaeckenstedt@tu-braunschweig.de
Jacob Müller	jacob.mueller@tu-braunschweig.de
Mohamed Amine Halila	m.halila@tu-braunschweig.de
Omar Zitouni	o.zitouni@tu-braunschweig.de

Braunschweig, 31. Mai 2023

Bearbeiterübersicht

Kapitel	Autoren	Kommentare
1	Jacob	...
1.1	Jacob	...
1.2	Jacob	...
1.3	Jacob	...
1.4	Jacob	...
2	Omar, Mohamed Amine	...
3	Omar, Mohamed Amine	...
4	Nazli, Keanu	...
5	Keanu	...
6	Mohamed Amine, Omar	...
6.1	Mohamed Amine, Omar	...
6.2	Mohamed Amine, Omar	...
6.3	Mohamed Amine, Omar	...
7	Keanu	...
8	Tristen	...
8.1	Tristen	...
8.2	Tristen	...
8.3	Tristen	...
9	Alle	...

Inhaltsverzeichnis

1 Zielbestimmung	5
1.1 Musskriterien	5
1.2 Sollkriterien	6
1.3 Kannkriterien	6
1.4 Abgrenzungskriterien	6
2 Produkteinsatz	7
2.1 Anwendungsbereiche	7
2.2 Zielgruppen	8
2.3 Betriebsbedingungen	8
3 Produktübersicht	9
4 Produktfunktionen	12
5 Produktdaten	22
6 Nichtfunktionale Anforderungen	23
6.1 Funktionalität	23
6.2 Sicherheit	23
6.3 Benutzbarkeit	24
6.4 Änderbarkeit	24
6.5 Qualitätsanforderungen	25
7 Benutzeroberfläche/Schnittstellen	26
8 Technische Produktumgebung	27
8.1 Software	27
8.2 Hardware	27
8.3 Produktschnittstellen	27
9 Glossar	28

Abbildungsverzeichnis

3.1	Use-Case-Diagramm: Lock Freie Datenstruktur Library	10
3.2	Visualisierung	11
7.1	Visualisierungsprogramm $\langle UI60 \rangle$	26

1 Zielbestimmung

Das effiziente und schnelle Arbeiten von Computern ist in der heutigen Zeit ein wichtiges Thema. Um möglichst schnell zu abreiten setzt man, dabei auf Multithreading, welches jedoch Probleme, wie Datenverlust durch das ABA-Problem, mit sich bringt.

Zum arbeiten mit Multithreading auf Datenstrukturen gibt es verschiedene Ansätze, den lock- und den lockfreien Ansatz.

Um die lockfreien Datenstrukturen soll es in diesem Softwareprojekt gehen, das Ziel ist es eine C-Library für diese Datenstrukturen zu entwickeln. Bei lockfreien Datenstrukturen gibt es den blocking- und den non-blocking Ansatz. Aufgrund der besseren Performance wird ein non-blocking Algorithmus entwickelt.

Durch die Verwendung von Pointern in lockfreien Algorithmen werden nach dem entfernen oder einfügen in Datenstrukturen die neuen Pointer bestimmt. Dazu wird das atomare Compare-AndSwap (CAS) verwendet, dabei werden zwei Speicherzellen/Pointer mit einander verglichen und wenn die Werte übereinstimmen wird ein dritter Wert in die erste Speicherzelle geschrieben. Durch eine derartige Programmierung der Algorithmen werden sichere lockfreie Datenstrukturen zur verfügung gestellt.

1.1 Musskriterien

Musskriterien: unabdingbare Leistungen der Software.

Hier wird aufgeführt, welche Funktionalitäten/Leistungen das Softwareprodukt in jedem Fall erfüllen muss, damit es genutzt werden kann.

- ⟨RM1⟩ Die Datenstrukturen müssen in der Programmiersprache C entwickelt werden
- ⟨RM2⟩ Die Software muss lockfrei sein.
- ⟨RM3⟩ Es dürfen keine Fehler beim einfügen von Elementen in die Datenstrukturen auftreten.
- ⟨RM4⟩ Es dürfen keine Fehler beim entfernen von Elementen aus der Datenstrukturen auftreten.
- ⟨RM5⟩ Die Software muss mit Multithreading zu recht kommen.
- ⟨RM6⟩ Es muss die atomic-Library verwendet werden.
- ⟨RM7⟩ Es dürfen keine Fehler beim Finden von Elementen im Set auftreten.

1.2 Sollkriterien

Sollkriterien: erstrebenswerte Leistungen

Dies sind Kriterien, die für die Lauffähigkeit des Produkts nicht zwingend erforderlich sind, für die Erreichung der Projektziele aber erfüllt werden sollten.

⟨RS1⟩ Als Compiler sollte clang verwendet werden.

⟨RS2⟩ Der Algorithmus sollte auf Windows, MAC OS und Linux funktionieren.

⟨RS3⟩ Die Software soll visualisiert werden.

⟨RS4⟩ Es soll gezeigt werden, wie schnell die Operationen auf den Datenstrukturen ausgeführt werden.

⟨RS5⟩ Als Datenstrukturen sollen Stack, Queue und List/Set implementiert werden.

1.3 Kannkriterien

Kannkriterien: Leistungen die enthalten sein können, denen der Auftraggeber jedoch neutral gegenüber steht.

⟨RC1⟩ Mehrere Versionen derselben Datenstruktur können implementiert werden. Beispielsweise Harris List und Micheals List.

1.4 Abgrenzungskriterien

Abgrenzungskriterien: Leistungen die explizit nicht umgesetzt werden.

Hier ist zu verdeutlichen, welche Ziele mit dem Produkt bewusst nicht erreicht werden sollen oder können.

⟨RW1⟩ Es wird kein Handbuch mitgeliefert.

⟨RW2⟩ Die Funktionen der Library sollen nicht optimiert sein.

⟨RW3⟩ Auf spezifische Hardware wird nicht geachtet.

2 Produkteinsatz

In diesem Abschnitt wird erläutert, wie das Produkt in die Zielumgebung integriert wird. Es werden nicht nur die Anwendungsbereiche und Zielgruppen beschrieben, die mit dem Produkt interagieren können, sondern es wird auch aufgezeigt, welche Voraussetzungen erfüllt sein müssen, um eine reibungslose Funktionsweise des Produkts sicherzustellen.

2.1 Anwendungsbereiche

Lock-Free Datenstrukturen kann in viele Anwendungen, wo mehrere Threads/Prozesse gleichzeitig auf gemeinsam genutzte Daten zugreifen und ändern, benutzt werden:

- Real-time systems: In Echtzeit-Systemen müssen Daten innerhalb bestimmter Zeitrahmen verarbeitet werden. Lock-Free-Datenstrukturen können dabei helfen, die Leistung und Geschwindigkeit der Datenverarbeitung zu erhöhen.
- Robotics: Lock-freie Datenstrukturen sind in der Robotik ein wichtiger Bestandteil zur Verwaltung von Sensordaten und der simultanen Steuerung von mehreren Roboteraktoren.
- Web development: Lock-free Datenstrukturen können in Web-Servern verwendet werden, um Anfragen und Antworten von mehreren Clients gleichzeitig zu verarbeiten.
- Online Gaming: In der Videospielentwicklung können Lock-free Datenstrukturen verwendet werden, um Eingaben von mehreren Spielern in Echtzeit zu verarbeiten.

Aber Lock-Free Datenstrukturen kann nicht für alle Nutzfälle verwendet werden zum beispiel:

- Limited hardware resources: Lock-free Datenstrukturen können mit für die Anwendung mit begrenzten Hardware-Ressourcen ungeeignet sein, weil sie mehr Speicher als traditionelle Datenstrukturen erfordern.
- Complex data structures: Lock-free Datenstrukturen können schwer zu implementieren sein, wenn die Datastruktur komplex ist.

2.2 Zielgruppen

Lock-Free-Datenstrukturen-Library ist im Wesentlichen für Entwickler von Anwendungen gedacht, die nebenläufige Operationen ausführen. Die können zum Beispiel Entwickler von Betriebssystemen, Real-Time Systemen, und Multicore-Systemen sein.

Für die andere Endanwender zum Beispiel Sekretärin sind Lock-free Datenstrukturen bedeutungslos, denn sie interagieren nicht direkt mit den Datenstrukturen, sondern nur die verwenden über eine grafische Benutzeroberfläche.

2.3 Betriebsbedingungen

- Multi-Core-Systeme: Das Gerät muss mehrere Prozessorkerne haben, auf denen parallel Aufgaben ausgeführt werden können.
- Speicher-Ressourcen: Das Gerät muss ausreichende Speicherkapazität haben, um die Lock-Free Datenstrukturen effektiv zu nutzen
- Hardwareunterstützung: Atomare Operationen müssen von der Hardware unterstützt sein.

3 Produktübersicht

Dieser Abschnitt hat die Aufgabe, die Funktionalität des zu entwickelnden Systems grafisch mit Hilfe von Use-Case-Diagrammen zu beschreiben. Abläufe sind in den Aktivitätsdiagrammen dargestellt. Alle Diagramme haben begleitende Texte.

Abbildung 3.1:

Im Use-Case-Diagramm „Lock Freie Datenstruktur Library“ erhält man einen Überblick über die Funktionalitäten der Bibliothek.

Da es sich bei diesem Projekt um eine verifizierte C-Bibliothek für lockfreie Datenstrukturen handelt, ist der Akteur der Entwickler, der die Funktionalitäten der Bibliothek nutzen wird. Der Akteur muss zuerst die gewählte Datenstruktur initialisieren. Danach hat er die Möglichkeit, (parallele) Einfüge- und/oder Löschooperationen darauf auszuführen. Bei Set kann der Akteur auch eine Find-Operation durchführen, um festzustellen, ob ein Element bereits im Set existiert oder nicht.

Abbildung 3.2:

In dem Use-Case-Diagramm „Visualisierung“ sieht man einen Überblick über den User Interface der Visualisierung. Der Akteur muss zunächst eine Datenstruktur und gültige Argumente wählen. Zum Beispiel müssen eine positive Anzahl von Threads sowie Einfüge- und Löschooperationen festgelegt werden. Danach, wenn der Akteur auf „Start“ klickt, startet die Visualisierung des Vergleichs zwischen lock-basierten und lockfreien Datenstrukturen und erhält der Akteur einen Graphen, der den Vergleich darstellt, wobei auf der X-Achse die Anzahl der Operationen und auf der Y-Achse die Zeit angegeben sind.

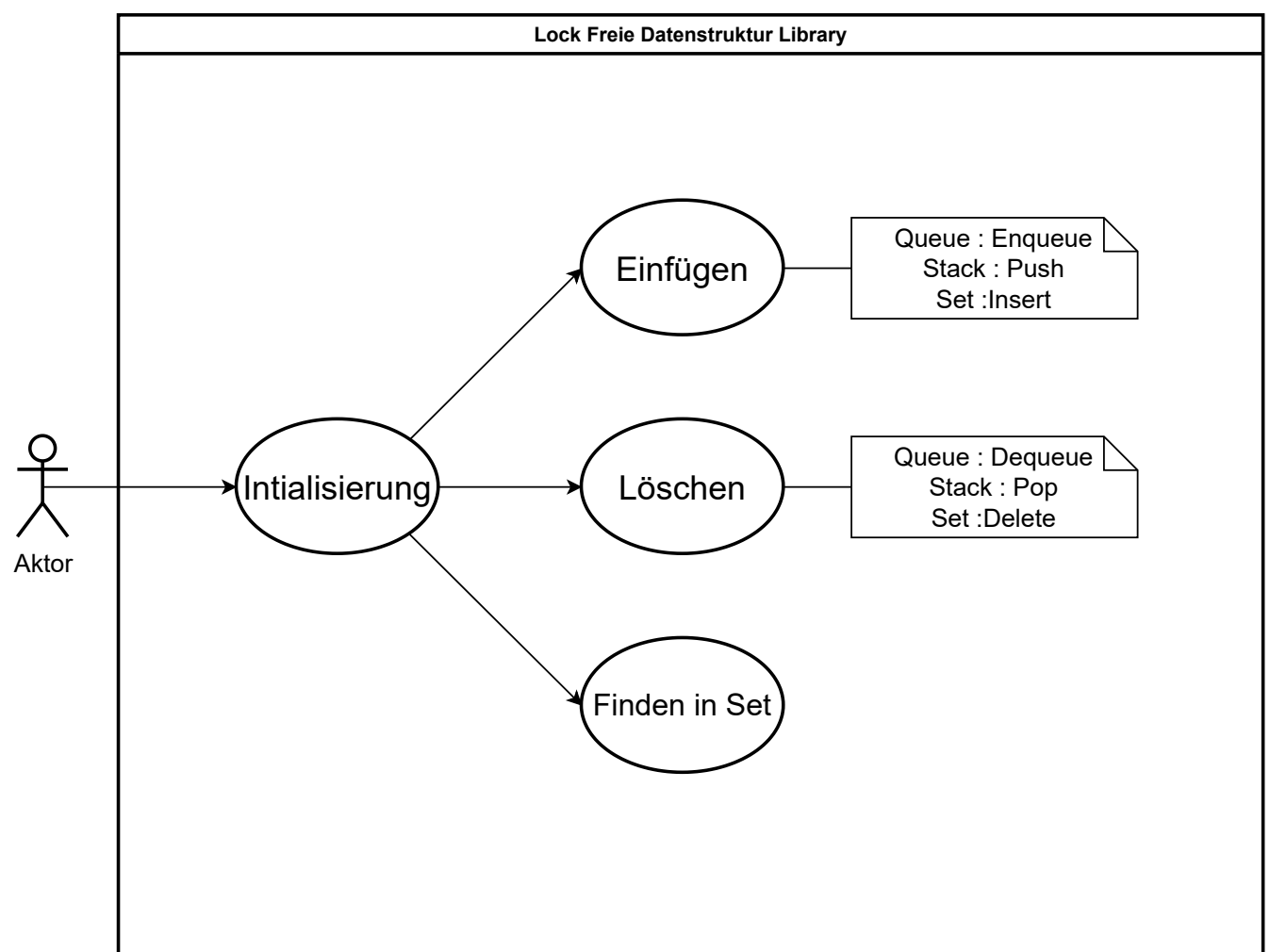


Abbildung 3.1: Use-Case-Diagramm: Lock Freie Datenstruktur Library

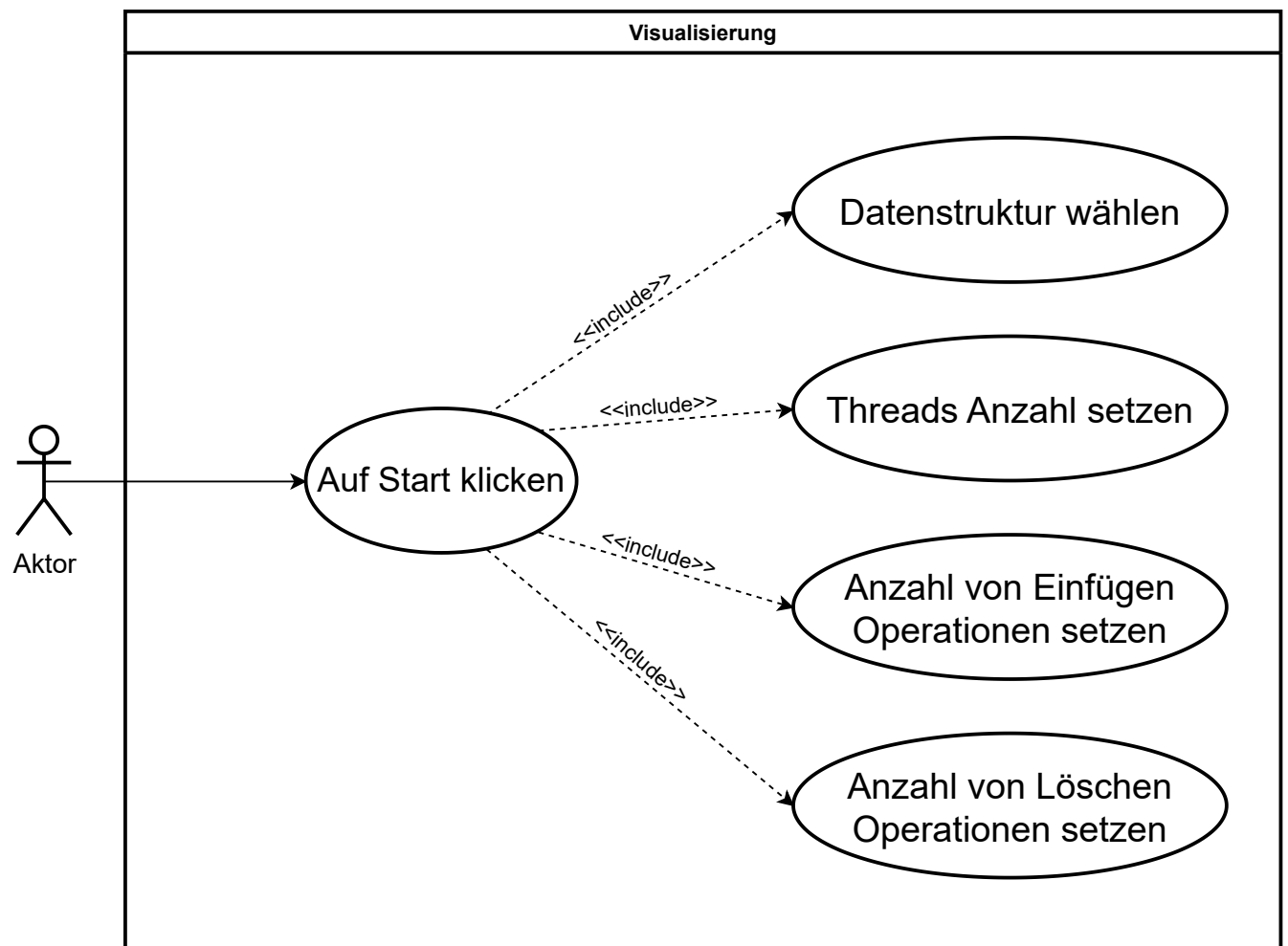


Abbildung 3.2: Visualisierung

4 Produktfunktionen

Pushen in Stack $\langle F10 \rangle$

Anwendungsfall: ein neues Element in den Stack hinzufügen

Anforderung: $\langle RM1 \rangle \langle RM2 \rangle \langle RM3 \rangle \langle RM4 \rangle \langle RM5 \rangle$

Ziel: Synchronisierte, datenverlustfreie und leistungsfähige Pushoperation mit mehreren Threads durchführen

Vorbedingung: Das Funktionsargument `stack*` zeigt auf einen Stack.

Nachbedingung Erfolg: Es wurde ein Node mit gewünschten Value erstellt und alloziert. TOS zeigt auf neues Element.

Nachbedingung Fehlschlag: Element konnte wegen der falschen Eingabe nicht gepusht werden

Akteure: Nutzer

Auslösendes Ereignis: Der Nutzer hat die Funktion Push aufgerufen.

Beschreibung:

1. Der Nutzer stellt gewünschten Value und den Stack bereit
2. Falls der gewünschte Value nicht geeignet ist, wird eine Fehlermeldung geworfen
3. Ein Element mit gegebenem Value erstellen
4. TOS zeigt auf neues Element

Poppen aus Stack $\langle F20 \rangle$

Anwendungsfall: letzt hinzugefügtes Element aus dem Stack entfernen

Anforderung: $\langle RM1 \rangle \langle RM2 \rangle \langle RM3 \rangle \langle RM4 \rangle \langle RM5 \rangle$

Ziel: Synchronisierte, datenverlustfreie und leistungsfähige Popoperation mit mehreren Threads durchführen

Vorbedingung: Das Funktionsargument `stack*` zeigt auf einen Stack.

Nachbedingung Erfolg: Die Node wurde erfolgreich entfernt und dealloziert. TOS zeigt auf das nächste Element von gepoppten Element.

Nachbedingung Fehlschlag: Element konnte wegen des leeren Stacks nicht gepoppt werden.

Akteure: Nutzer

Auslösendes Ereignis: Der Nutzer hat die Funktion Pop aufgerufen.

Beschreibung:

1. Falls Top auf NULL zeigt, wird eine Fehlermeldung geworfen
2. das letzt gepoppte Element bzw. Top wird aus dem Stack gelöscht
3. Top zeigt auf das nächste Element von gepoppten Element

Erweiterung: (optional)

3. Falls Stack nur aus einem Element besteht, zeigt sowohl Top auf NULL

Enqueue $\langle F30 \rangle$

Anwendungsfall: ein neues Element in Queue hinzufügen

Anforderung: $\langle RM1 \rangle$ $\langle RM2 \rangle$ $\langle RM3 \rangle$ $\langle RM4 \rangle$ $\langle RM5 \rangle$

Ziel: Synchronisierte, datenverlustfreie und leistungsfähige Enqueue Operation mit mehreren Threads durchführen

Vorbedingung: Das Funktionsargument Q^* zeigt auf einen Queue.

Nachbedingung Erfolg: Es wurde ein Node mit gewünschten Value erstellt und alloziert. $Tail^*$ zeigt auf neues Element

Nachbedingung Fehlschlag: Element konnte wegen der falschen Eingabe nicht hinzugefügt werden.

Akteure: Nutzer

Auslösendes Ereignis: Der Nutzer hat die Funktion Enqueue aufgerufen

Beschreibung:

1. Falls der gewünschte Value nicht geeignet ist, wird eine Fehlermeldung geworfen
2. Element wird mit gegebenem Value erstellt
3. $Tail^*$ zeigt auf das hinzugefügte Element

Erweiterung: (optional)

3. Wenn ein anderer Thread gleichzeitig ein Element hinzugefügt hat bzw. wenn $Tail^*$ nicht auf letzte Node zeigt, wird der Tail als das letzte Element, das der andere Thread hinzugefügt hat, aktualisiert.

Dequeue $\langle F40 \rangle$

Anwendungsfall: erst hinzugefügtes Element aus Queue entfernen

Anforderung: $\langle RM1 \rangle \langle RM2 \rangle \langle RM3 \rangle \langle RM4 \rangle \langle RM5 \rangle$

Ziel: Synchronisierte, datenverlustfreie und leistungsfähige Dequeue Operation mit mehreren Threads durchführen

Vorbedingung: Das Funktionsargument Q^* zeigt auf einen Queue

Nachbedingung Erfolg: Der Node wurde erfolgreich entfernt und dealloziert. Head zeigt auf das nächste Element vom entfernten Element

Nachbedingung Fehlschlag: Operation hat kein Erfolg wegen der leeren Queue

Akteure: Nutzer

Auslösendes Ereignis: Der Nutzer hat die Funktion Dequeue aufgerufen

Beschreibung:

1. Falls Head auf NULL zeigt, wird eine Fehlermeldung geworfen
2. das Element wird aus der Queue entfernt
3. Head zeigt auf das nächste Element von entfernten Element

Erweiterung: (optional)

3. Falls Queue nur aus einem Element besteht, zeigt sowohl Head als auch Tail auf NULL

Insert in Set $\langle F50 \rangle$

Anwendungsfall: ein neues Element korrekt in das Set einfügen

Anforderung: $\langle RM1 \rangle$ $\langle RM2 \rangle$ $\langle RM3 \rangle$ $\langle RM4 \rangle$ $\langle RM5 \rangle$

Ziel: Synchronisierte, datenverlustfreie und leistungsfähige Insert Operation mit mehreren Threads durchführen

Vorbedingung: Es liegt eine schon initialisiertes Set vor, und das Funktionsargument `list*` zeigt auf dieses.

Nachbedingung Erfolg: Es wurde eine Node mit gewünschten Key erstellt und alloziert, dieses soll sich auch an richtiger Stelle im Set befinden.

Nachbedingung Fehlschlag: Bei Fehlschlag soll das Objekt Dealloziert werden und die Liste Intakt bleiben.

Akteure: Nutzer

Auslösendes Ereignis: Der Nutzer ruft die Funktion `insert` auf.

Beschreibung:

1. Der Nutzer stellt gewünschten Key und das Set bereit
2. Ein Element mit gegebenem Key wird erstellt
3. Position im Set wird ermittelt
4. Neue Node wird mit CAS eingefügt

Delete in Set $\langle F60 \rangle$

Anwendungsfall: ein neues Element korrekt aus dem Set entfernen

Anforderung: $\langle RM1 \rangle$ $\langle RM2 \rangle$ $\langle RM3 \rangle$ $\langle RM4 \rangle$ $\langle RM5 \rangle$

Ziel: Synchronisierte, datenverlustfreie und leistungsfähige Deleteoperation mit mehreren Threads durchführen

Vorbedingung: Es liegt ein schon initialisiertes Set vor, und das Funktionsargument `list*` zeigt auf dieses.

Nachbedingung Erfolg: Es wurde ein Node mit gewünschtem Key gefunden, logisch entfernt und eventuell dealloziert.

Nachbedingung Fehlschlag: Bei Fehlschlag soll die Liste intakt bleiben.

Akteure: Nutzer

Auslösendes Ereignis: Der Nutzer ruft die Funktion `delete` auf.

Beschreibung:

1. Der Nutzer stellt gewünschten Key und das Set bereit
2. Ein Element mit gegebenem Key wird gesucht
3. Falls gefunden wird das Element mit CAS logisch gelöscht (markiert)
4. Search wird ausgeführt falls hardware löschen noch nicht ausgeführt

Find in Set $\langle F70 \rangle$

Anwendungsfall: Überprüfe ob Element im Set vorhanden ist.

Anforderung: $\langle RM1 \rangle$ $\langle RM2 \rangle$ $\langle RM3 \rangle$ $\langle RM4 \rangle$ $\langle RM5 \rangle$

Ziel: Synchronisierte, datenverlustfreie und leistungsfähige Findoperation mit mehreren Threads durchführen.

Vorbedingung: Es liegt eine schon initialisiertes Set vor, und das Funktionsargument `list*` zeigt auf dieses.

Nachbedingung Erfolg: Es wurde `true` oder `false` zurückgegeben je nachdem ob das Element im Set vorhanden ist.

Nachbedingung Fehlschlag: Bei Fehlschlag soll die Liste Intakt bleiben.

Akteure: Nutzer

Auslösendes Ereignis: Der Nutzer ruft die Funktion `find` auf.

Beschreibung:

1. Der Nutzer stellt gewünschten Key und das Set bereit
2. Search wird mit gewünschtem Key durchgeführt
3. Ergebnis aus Search wird überprüft
4. Gebe Ergebnis zurück

Search in Set $\langle F80 \rangle$

Anwendungsfall: Ermittle linkes und rechtes Element für gegebenen Key.

Anforderung: $\langle RM1 \rangle$ $\langle RM2 \rangle$ $\langle RM3 \rangle$ $\langle RM4 \rangle$ $\langle RM5 \rangle$

Ziel: Synchronisierte, datenverlustfreie und leistungsfähige Searchoperation mit mehreren Threads durchführen.

Vorbedingung: Es liegt eine schon initialisiertes Set vor, und das Funktionsargument `list*` zeigt auf dieses.

Nachbedingung Erfolg: `Leftnode*` wurde auf linkes Node gesetzt und `right node` zurückgegeben. Außerdem wurden alle logisch gelöschten Nodes zwischen `left` und `right` entfernt.

Nachbedingung Fehlschlag: Bei Fehlschlag soll die Liste Intakt bleiben.

Akteure: Andere Set funktionen

Auslösendes Ereignis: Der Nutzer ruft die Funktion `insert`, `delete` oder `find` auf.

Beschreibung:

1. Es wird ein Nodezeiger und der gewünschte Key übergeben
2. Left und right Node werden anhand des Keys ermittelt und `leftnode*` auf left Node gesetzt
3. Es wird überprüft ob logisch gelöschte Nodes zwischen left und right Node liegen
4. Lösche und dealloziere logisch gelöschte Nodes zwischen left und right Node
5. Gebe right Node zurück

Erweiterung: Insert, Delete und Find implementieren Search

Benchmark Visualisieren $\langle F90 \rangle$

Anwendungsfall: Biete UI und Grafik für Effizienzunterschied zwischen Lock-free und lock-based

Anforderung: $\langle RM1 \rangle$ $\langle RM2 \rangle$ $\langle RM3 \rangle$ $\langle RM4 \rangle$ $\langle RM5 \rangle$

Ziel: Stelle Unterschied der Leistung grafisch dar.

Vorbedingung: Programm ist gestartet und Parameter sind eingegeben

Nachbedingung Erfolg: Eine Grafik entsprechend der Parameter wurde erstellt und wird dem User präsentiert.

Nachbedingung Fehlschlag: Bei Fehlschlag soll eine Fehlermeldung ausgegeben werden.

Akteure: Nutzer

Auslösendes Ereignis: Der Nutzer betätigt den start Knopf

Beschreibung:

1. Programm wird gestartet
2. Parameter (Threadanzahl, Anzahl Zugriffe und Datenstruktur) werden ausgewählt
3. Startknopf wird betätigt
4. Benchmark liefert Ergebnisse
5. Ergebnisse werden Visualisiert

Erweiterung: Das Programm ruft einen C-Benchmark aus der auf die Library zugreift.

Benchmark $\langle F100 \rangle$

Anwendungsfall: Ermittle Effizienzunterschied zwischen Lock-free und lock-based mithilfe von C-Skript

Anforderung: $\langle RM1 \rangle$ $\langle RM2 \rangle$ $\langle RM3 \rangle$ $\langle RM4 \rangle$ $\langle RM5 \rangle$

Ziel: Liefere Daten für den Leistungsunterschied.

Vorbedingung: Benchmark wird aufgerufen

Nachbedingung Erfolg: Für Visualisierung erforderliche Daten werden an das Visualisierungsprogramm weitergegeben.

Nachbedingung Fehlschlag: Bei Fehlschlag soll eine Fehlermeldung ausgegeben werden.

Akteure: UI

Auslösendes Ereignis: Der Nutzer betätigt den Start Knopf in der UI

Beschreibung:

1. Starte Programm
2. Erstelle gewünschte Anzahl an Threads
3. Starte Overall Timer
4. Starte für jeden Thread einzeln Zugriff Timer
5. Betätige Zugriff
6. Stoppe Zugriffs Timer
7. Wiederhole Schritt 4 bis 6 bis gewünschte Anzahl an Zugriffe erfolgt ist
8. Stoppe Overall Timer
9. Berechne Mittelwert einzelner Zugriffe
10. Liefere Gesamtzeit und Durchschnittszeit an Visualisierung zurück

Erweiterung: Das Programm ruft einen C-Benchmark aus der auf die Library zugreift.

5 Produktdaten

Unsere Library benötigt keine längerfristige Speicherung von Daten, lediglich die Key Values der einzelnen Einheiten werden gespeichert und bei Programmende wieder entfernt. Auch keine Diagnosedaten oder Systeminformationen müssen gesammelt werden. Bei der Visualisierung werden die Ergebnisse des Benchmarks direkt an das zuständige Programm weitergegeben, es findet also auch hier keine längerfristige Speicherung von Daten statt.

6 Nichtfunktionale Anforderungen

In diesem Kapitel wird festgelegt, welche Qualitätsmerkmale das zu entwickelnde Produkt in welcher Qualitätsstufe besitzen soll. Anschließend werden die als am wichtigsten bezeichneten Qualitätsmerkmale operationalisiert, d.h. in konkrete Produktanforderungen detailliert, falls sie nicht als allgemeine Richtlinie zur Verfügung gestellt werden können.

6.1 Funktionalität

Produktqualität	sehr gut	gut	normal	nicht relevant
Angemessenheit	x			
Richtigkeit	x			
Interoperabilität			x	
Ordnungsmäßigkeit	x			

Angemessenheit, Richtigkeit und Ordnungsmäßigkeit: sehr gut denn die Lock-freie-Datenstrukturen-Bibliothek soll in der Lage sein, die Funktionen angemessen, fehlerfrei und korrekt auszuführen & effizient zu arbeiten.

Interoperabilität: normal denn wir müssen nur Lock-freie-Datenstrukturen-C-Bibliothek implementieren und es war nicht erforderlich, dass sie mit spezifischen Systeme interoperabel ist.

6.2 Sicherheit

Produktqualität	sehr gut	gut	normal	nicht relevant
Zuverlässigkeit	x			
Reife	x			
Fehlertoleranz		x		
Wiederherstellbarkeit			x	

Zuverlässigkeit: sehr gut denn das Leistungsniveau der Bibliothek wird sowohl von unserem Gruppe als auch von Gruppe 3 optimiert.

Reife: sehr gut weil die Bibliothek sowohl von unserem Gruppe als auch von Gruppe 3 getestet wird.

Fehlertoleranz: gut denn wenn es Fehler gibt, soll es Fehlermeldung geben.

Wiederherstellbarkeit: normal denn bei einem Ausfall des Systems, können die Daten verloren gehen.

6.3 Benutzbarkeit

Produktqualität	sehr gut	gut	normal	nicht relevant
Verständlichkeit		x		
Erlernbarkeit		x		
Bedienbarkeit		x		
Effizienz		x		
Zeitverhalten	x			
Verbrauchsverhalten		x		

Verständlichkeit, Bedienbarkeit, Erlernbarkeit und Effizienz: gut denn Entwickler, die die Lock-Freie-Datenstrukturen-Bibliothek verwenden werden, soll schnell die Bibliothek verstehen ,effektiv erlernen und nutzen können.

Zeitverhalten: sehr gut weil die Lock-Free Datenstrukturen schnell sein soll, um parallele Ausführung zu ermöglichen und somit die Leistung zu verbessern.

Verbrauchsverhalten: gut denn Die Lock-Free Datenstrukturen Bibliothek kein grosses Verbrauchsverhalten hat.

6.4 Änderbarkeit

Produktqualität	sehr gut	gut	normal	nicht relevant
Analysierbarkeit		x		
Modifizierbarkeit		x		
Stabilität		x		
Prüfbarkeit		x		
Übertragbarkeit		x		
Anpassbarkeit		x		
Installierbarkeit		x		
Konformität		x		
Austauschbarkeit			x	

Analysierbarkeit, Modifizierbarkeit, Stabilität, Prüfbarkeit und Konformität: gut denn die Lock-Free Datenstruktur Bibliothek wird gut strukturiert, leicht verständlich, modifizierbar und prüfbar sein.

Übertragbarkeit und Anpassbarkeit: gut weil die Bibliothek keine spezifischen Abhängigkeiten haben muss, die nur auf bestimmten Umgebung vorhanden sind.

Installierbarkeit: gut, da der Benutzer die Bibliothek einfach in seiner Entwicklungsumgebung installieren kann.

Austauschbarkeit: normal weil das abhängig von der Hardware sein kann, zum beispiel ob die Atomare Operationen unterstützt sind oder nicht.

6.5 Qualitätsanforderungen

Die oben als am wichtigsten bezeichneten Qualitätsmerkmale werden im Folgenden operationalisiert, d.h. in konkrete Produktanforderungen detailliert oder es wird angegeben, welche Richtlinie einzuhalten ist.

- $\langle Q10 \rangle$ Das Produkt soll betriebssystemunabhängig sein
- $\langle Q20 \rangle$ Der Code soll gut kommentiert und übersichtlich sein
- $\langle Q30 \rangle$ Das Produkt soll leicht zur Benutzung sein
- $\langle Q40 \rangle$ Das Produkt soll schneller als ähnliche Produkte die lock freie Mechanismus benutzen
- $\langle Q50 \rangle$ Das Produkt soll kontinuierlich getestet werden
- $\langle Q60 \rangle$ Die Sprache des Produktes soll Englisch sein

7 Benutzeroberfläche/Schnittstellen

Benutzeroberfläche:

Für die Library wird es keine UI geben, da diese einfach in ein C-Programm importiert wird und die Funktionen dann im Programm verwendbar sind (siehe Schnittstellen für mehr). Für die Visualisierung wird es allerdings eine UI geben die sich der Funktion $\langle F60 \rangle$ bedient.

Das Programm läuft mit non Admin rechten und es gibt keine Einstufung der Funktionen in Berechtigungen.

Visualisierungsprogramm $\langle UI60 \rangle$

In Number Threads kann man einstellen mit wie vielen Threads getestet wird. In Number Insertions und Number Deletions werden die Anzahl der gewünschten Zugriffe eingestellt. Select Data Structure stellt die 3 möglichen Data strutures zur Auswahl. Sobald alles eingestellt lässt sich das Programm mit Start Knopf starten.

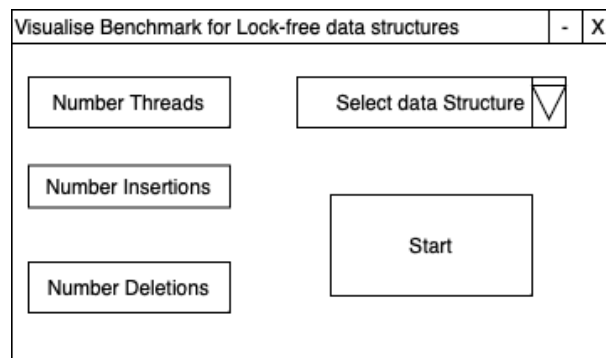


Abbildung 7.1: Visualisierungsprogramm $\langle UI60 \rangle$

Schnittstellen:

Alle funktionen der Library können in gewöhnlichen C-Programmen importiert und dann verwendet werden. Dies ist die einzige möglichkeit wie man auf die Funktionen zugreifen kann.

8 Technische Produktumgebung

Die gelieferte C-Library wird auf den, in diesem Kapitel beschriebenen, Betriebssystemen und der folgenden Hardware ausführbar und nutzbar sein.

8.1 Software

Die zu entwickelnde Software läuft auf den Echtzeitbetriebssystemen Linux, macOS und Windows. Als Entwicklungsumgebung (IDE) wird CLion verwendet, als Build-Tool wird CMake genutzt, sowie ist die Implementierungssprache C.

8.2 Hardware

Die Library kann auf allen Hardwaresystemen ausgeführt werden die C unterstützen.

8.3 Produktschnittstellen

Das Produkt soll in neuen oder bestehenden C-Projekten genutzt werden, dazu bekommt die Software eine API-Schnittstelle damit jedes Programm die Library inkludieren kann.

9 Glossar

Hier werden Fachbegriffe erklärt.

Bearbeiterübersicht

Wörter	Kommentare
ABA-Problem	Ein Datenbankproblem, bei dem ein Wert zweimal gelesen wird und beide mal den gleichen Wert zurück liefert
API - Application Programming Interface	eine Sammlung von definierten Methoden, Funktionen, Protokollen und Werkzeugen
Blocking	Technik, um parallele Prozesse zu synchronisieren
Build-Tool	Softwareanwendung die ermöglicht das Kompilieren und Zusammenfügen des Quellcodes zu steuern
CompareAndSwap (CAS)	Operation um Locking-, Synchronisationsoperationen zu implementieren
Lock	Zugriffsmöglichkeit auf Datenbanken beim Multithreading
Lockfrei	Zugriffsmöglichkeit auf Datenbanken beim Multithreading
Multithreading	gleichzeitige Bearbeitung mehrerer Prozesse
non blocking	Technik um parallele Prozesse zu synchronisieren
Pointer	Objekt, das eine Speicheradresse zwischenspeichert
TOS/Top	Top of Stack, Zeiger, der das letzt hinzugefügte Element in Stack zeigt
Tail	Zeiger, der das letzt hinzugefügte Element in Queue zeigt
Head	Zeiger, der das erst hinzugefügte Element in Queue zeigt
Thread	Federgewichtiger Prozess mit geteiltem Speicher.
Node	Objekt innerhalb der Datenstruktur.
Key	Wert einer Node.
alloziert	Speicher wird auf dem Heap belegt und Objekt dort eingelagert.
dealloziert	Speicher wird auf dem Heap freigegeben.
logisch entfernt	Objekt wird als "entfernt" markiert aber nicht real ausgehängt.
UI	User Interface.